

Étude comparative pour l'ordonnancement industriel (JSSP) : Solveurs exacts vs Apprentissage par Renforcement

Analyse de performance, modélisation des récompenses et limites structurelles des architectures denses (MLP) face à la généralisation.

Bob Lenglet, le 14/06/2026

Github : <https://github.com/bobL59>

Résumé

Cette étude compare un solveur exact (OR-Tools) à une approche RL (MaskablePPO) sur l'instance JSSP FT10. Bien que le modèle RL peine à atteindre l'optimalité (2608 u.t contre 930 u.t) en raison des limites géométriques de l'architecture MLP, il permet un réajustement en temps réel, ouvrant la voie à des futures implémentations basées sur les graphes (GNN).

1 : Introduction et Cadrage Métier

1.1 Le défi industriel du Job Shop Scheduling Problem (JSSP)

Le Job Shop Scheduling Problem (JSSP) est un problème d'optimisation NP-difficile impliquant N commandes (composées de plusieurs tâches) devant passer sur M machines dans un ordre précis. L'objectif est de terminer toutes ces commandes en un temps minimum, ce que nous appelons le Makespan.

Dans une usine, le temps, c'est de l'argent. Une machine qui tourne à vide ou la formation d'un goulot d'étranglement rallongent le temps total de production. Résoudre le JSSP en respectant les contraintes physiques d'une usine permet donc d'optimiser directement sa rentabilité. C'est tout l'enjeu de ce projet.

1.2 Objectif de l'étude : Réactivité vs Optimalité.

Face au JSSP, deux approches s'affrontent : la recherche de l'optimalité ou le besoin de réactivité.

D'un côté, les méthodes mathématiques classiques promettent des résultats parfaits pour une usine statique figée, mais elles nécessitent des temps de calcul importants. De l'autre côté, la réalité du terrain nous montre que les usines ne sont presque jamais

statiques. Une machine peut tomber en panne, une commande urgente peut s'intercaler, ou les plans peuvent changer à la dernière minute.

Dans ces situations, nous n'avons tout simplement pas le temps qu'un solveur recalcule entièrement une nouvelle solution, car chaque minute d'arrêt coûte très cher. C'est pourquoi, dans cette étude, je vais évaluer si une intelligence artificielle (RL) est capable d'apprendre des règles empiriques assez solides pour réajuster un planning en temps réel.

2 : La Baseline Déterministe : RO

2.1 Modélisation mathématique du problème.

La modélisation mathématique du JSSP repose sur un ensemble de contraintes strictes que le solveur doit respecter pour garantir une solution valide :

- Une machine ne peut traiter qu'une tâche à la fois.
- Les tâches d'une même commande ont un ordre de passage strict.
- Dans le cadre de l'instance FT10, chaque commande (job) doit obligatoirement passer par l'intégralité des machines.
- Une fois qu'une tâche a démarré sur une machine, elle ne peut pas être interrompue ou mise en pause avant son achèvement.
- Toutes les machines sont disponibles à l'instant $t = 0$ et les durées de transfert d'une machine à l'autre sont considérées comme nulles.

Maintenant que les contraintes du problème sont posées, nous pouvons le résoudre.

2.2 Résolution via Google OR-Tools et performance sur l'instance FT10.

Google OR-Tools s'est imposé comme un standard de l'industrie en matière d'optimisation. Il regroupe plusieurs algorithmes de Recherche opérationnelle, dont des solveurs par programmation par contraintes particulièrement efficaces pour le JSSP.

Pour évaluer les performances de notre approche, cette étude s'appuiera sur la célèbre instance Fisher et Thompson 10x10 (nommée FT10). Cette configuration historique a été conçue spécifiquement par les deux mathématiciens pour offrir un haut niveau de complexité et piéger les heuristiques classiques.

Face à ce défi, le solveur exact trouve la solution parfaite : un Makespan de 930 unités de temps (u.t). Ce score constitue le Makespan indépassable de notre projet. Le diagramme de Gantt ci-dessous illustre l'organisation millimétrée de cette solution optimale.

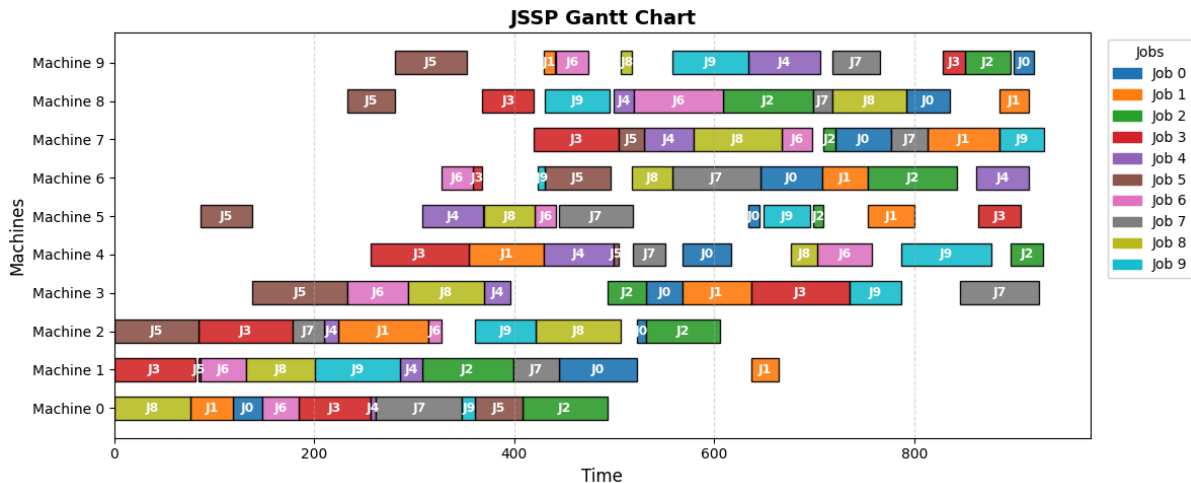


Figure 1 : Diagramme de Gantt de la solution optimale (930 u.t) générée par OR-Tools sur l'instance FT10.

3 : L'Approche par Renforcement Learning

3.1 Pourquoi le RL ? Le besoin de réactivité en environnement dynamique.

Comme expliqué précédemment, chercher la solution parfaite avec un solveur mathématique exige un temps de calcul qui peut rapidement devenir un problème face à la complexité d'une usine. C'est un délai que l'on ne peut pas se permettre sur le terrain en cas de panne ou d'imprévu.

C'est ici qu'intervient le Reinforcement Learning. L'avantage majeur de l'IA est que la quasi-totalité de la lourde charge de calcul est déportée sur la phase d'entraînement. Une fois cette phase terminée, le modèle agit comme un simple moteur capable de générer un planning valide en quelques millisecondes.

De ce fait, pour répondre à un besoin de réactivité, l'approche RL accepte de sacrifier l'optimalité au profit d'une solution instantanée. Le véritable défi de cette modélisation est donc le suivant : réussir à rendre cette approximation suffisamment performante pour qu'elle devienne viable en production.

3.2 Modélisation de l'environnement (Gymnasium et MaskablePPO).

Pour l'environnement de l'IA, nous utiliserons le modèle MLP (Multi Layer Perceptron), de ce fait, nous allons utiliser un vecteur d'entrée de taille fixe et d'une seule dimension (1D). Dans ce vecteur, il sera stocké :

- 1 - Les temps de disponibilités de chaque machine,
- 2 - Les temps de disponibilités de chaque pièce,
- 3 - Une anticipation $N + 1$: Pour chaque job, on récupère sa durée et la machine requise.

Pour permettre de garder des valeurs entre 0 et 1, les durées sont normalisées (divisées par le temps total aussi appelé horizon) et les identifiants de machines sont encodés avec le One-Hot encoding.

À chaque pas de temps t , l'agent sélectionne une action a_t correspondant à l'identifiant de la prochaine commande à planifier. L'espace d'action est donc défini comme un espace discret $A = \{0, 1, \dots, N - 1\}$, où N représente le nombre total de commandes de l'instance.

Afin d'accélérer l'entraînement, nous utiliserons un algorithme d'apprentissage de type Maskable PPO. De ce fait, les actions illégales sont impossibles et donc ne ralentissent pas l'entraînement.

Le tableau suivant regroupe les hyperparamètres et la configuration structurelle retenus pour cet entraînement :

Paramètre	Valeur / Configuration
Algorithme	Maskable PPO
Type de politique	MlpPolicy
Architecture du réseau	Acteur : [256, 256] Critique : [256, 256]
Taux d'apprentissage	3e-4
Coefficient d'entropie	0.00 ou 0.05 selon les scénarios (expliqué par le suite)

Figure 2 : Tableau synthétique des hyper-paramètres et la configuration structurelle du modèle

3.3 Les pièges de l'implémentation : Naive Reward et Sparse Reward.

Notre première idée de méthode de récompense a été la "Naive Reward". L'idée était de punir l'agent par la nouvelle date de fin de la tâche (-end_time). Cette approche a rapidement montré ses limites car l'agent plaçait les tâches courtes le plus rapidement possible, créant d'énormes goulots d'étranglement dans le futur.

La seconde approche a évidemment été la "Sparse Reward". Mais l'agent ne recevant un score qu'à la fin de l'épisode, il n'arrivait pas à comprendre quelle action précise l'avait mis en échec.

Ces deux approches de récompenses basiques ayant échoué, nous avons donc dû adapter la récompense nous-mêmes pour la suite du projet.

4 : Optimisation et Alignement des Objectifs

4.1 Le Reward Shaping : Introduction du Delta Makespan pour capter le temps masqué.

Nous avons donc pensé à une méthode de récompense différente : utiliser la différence de Makespan entre l'instant T et l'instant $T + 1$ (le Delta Makespan). Cette approche permet de capter le concept de "temps masqué". De cette manière, l'agent est capable de comprendre qu'il doit paralléliser les tâches au maximum pour ne pas faire augmenter le temps global, et ainsi obtenir une meilleure récompense.

```
makespan_before = max(self.machine_available_time)
# ... Mise à jour en fonction de l'action choisie
makespan_after = max(self.machine_available_time)
reward = -float(makespan_after - makespan_before)
```

4.2 Analyse de la convergence.

Observons maintenant les courbes d'apprentissage :

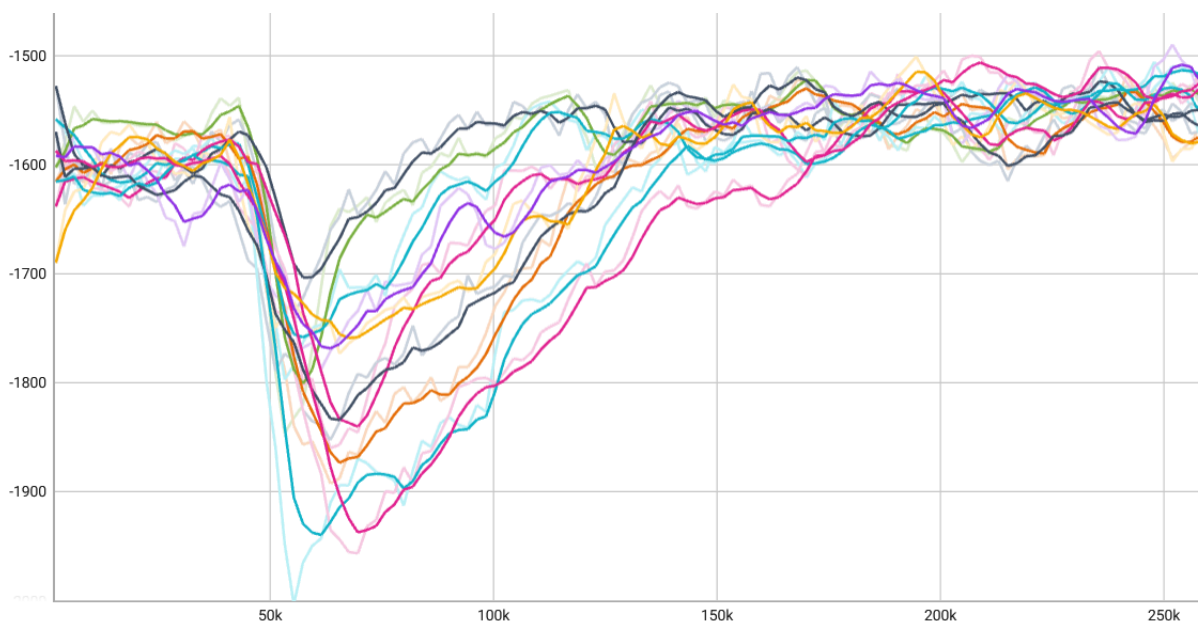


Figure 2 : Évolution de la récompense moyenne (ep_rew_mean) sur les 10 entraînements.

Sur ces courbes, nous pouvons distinguer un schéma qui se répète en trois phases :

- De 0 à ~60k étapes : une exploration hasardeuse.
- De ~60k à ~175k étapes : la compréhension du temps masqué.
- De ~175k à 400k étapes : l'atteinte d'un minimum local (stagnation).

La récompense sur l'axe des ordonnées n'est pas le résultat exact sur l'instance FT10, mais il s'agit d'un avant-goût plutôt clair. Nous remarquons nettement que les modèles stagnent et ne s'approchent pas de l'optimum de 930 u.t.

5 : Crash Test (FT10), Limites de l'Architecture et Perspectives

5.1 Confrontation de l'agent généraliste à l'instance de référence FT10 et impact de l'entropie.

Notre premier essai a été peu concluant. Le modèle stagnait au niveau de son minimum local. Nous avons donc eu une idée : si le modèle est bloqué ici, forçons-le à explorer davantage. Avec ça est venue l'idée d'intégrer une entropie.

Un premier test avec une entropie de 0.05 s'est avéré concluant (Makespan réduit d'environ 20 u.t sur la FT10). En revanche, ce résultat pouvait très bien être le fruit du hasard lors de l'entraînement. C'est pour ça que nous avons lancé une série de tests : 5 entraînements avec une entropie nulle et 5 autres entraînements avec une entropie de 0.05. Voici le tableau des résultats sur la FT10 :

	Entropy	Makespan
Entraînement		
Entraînement 1	0.00	2666.0
Entraînement 2	0.00	2368.0
Entraînement 3	0.00	2619.0
Entraînement 4	0.00	2382.0
Entraînement 5	0.00	3113.0
Entraînement 6	0.05	2441.0
Entraînement 7	0.05	2667.0
Entraînement 8	0.05	2774.0
Entraînement 9	0.05	2732.0
Entraînement 10	0.05	2430.0

Figure 3 : Tableau des résultats des 10 modèles sur la FT10 en fonction de leur entropie

Couplé à ces résultats, un calcul de moyenne et d'écart-type a été fait :

La solution optimale du ft10 est de **930 u.t**
Notre modèle sans entropie a trouvé une solution en moyenne de **2629.60 +/- 302.02 u.t**
Notre modèle avec une entropie de 0.05 a trouvé une solution en moyenne de **2608.80 +/- 162.77 u.t**

Nous remarquons qu'avec simplement le changement d'un hyper-paramètre, nous avons réussi à réduire le temps global d'environ 20 u.t, mais surtout, l'écart-type a été divisé par presque 2. Nous obtenons donc un modèle beaucoup plus robuste grâce à son exploration plus profonde.

Au niveau du temps d'inférence, nous sommes en moyenne à 0.68 ms. La promesse de réactivité est atteinte.

5.2 Analyse de la stagnation : La malédiction de la dimensionnalité du vecteur plat 1D.

Le problème majeur de nos modèles, et la cause principale de cette stagnation, réside dans le type de réseau de neurones utilisé. En effet, le réseau "MLP" s'apparente à une grande boîte rigide. Il prend une simple liste de nombres, mais ne comprend pas naturellement les liens de dépendance (par exemple, que la tâche A est physiquement bloquée par la tâche B). Sans cette logique relationnelle, il n'arrive pas à généraliser son comportement face à la structure complexe de l'instance finale, la FT10.

De plus, cette architecture nous empêche de mettre en place une approche progressive de la difficulté (Curriculum Learning). Nous ne pouvons pas commencer par lui donner des problèmes 3x3, puis 5x5, puis 8x8, pour l'amener doucement vers le 10x10. Pourquoi ? Car le vecteur d'entrée d'un réseau "MLP" possède une taille strictement fixe. Dans notre configuration, l'agent est mathématiquement contraint et ne peut traiter qu'une seule dimension d'instance à la fois.

5.3 Perspectives : Vers la modélisation en graphes disjonctifs et architectures GNN.

L'évolution logique de ce projet consisterait à remplacer notre vecteur 1D par une structure en graphe disjonctif, où les nœuds représentent les tâches et les arêtes des contraintes (de précédence et de machines). Il faudrait pour cela s'orienter vers une architecture GNN (Graph Neural Network). Avec un tel réseau, l'agent serait enfin en capacité de :

- Comprendre naturellement les dépendances physiques entre les tâches.
- Avoir une vue globale et structurée sur l'ensemble de l'usine.
- Être entraîné avec une difficulté progressive, la taille d'un graphe n'étant pas fixe en entrée (Curriculum Learning).

Bien qu'une telle implémentation nécessite plusieurs mois de recherche et de développement, cette étude m'aura permis de cartographier précisément le point de rupture des modèles classiques. En confrontant la rigueur des mathématiques exactes (RO) à l'adaptabilité de l'Apprentissage par Renforcement, ce projet illustre ma volonté d'aborder les défis industriels non pas avec des dogmes technologiques, mais avec un pragmatisme d'ingénieur centré sur la recherche de la solution la plus viable.

Github : <https://github.com/bobL59>